

Mycellius — Séance 10-11-12 (4h)

Industrialisation + durcissement + déploiement + preuve CIA

Objectif “produit” (ce qu’on livre à la fin)

- 1. Traçabilité** : révisions + audit “qui a fait quoi” (preuves consultables).
- 2. 1er durcissement** : validations renforcées + security headers + mitigation XSS Markdown + mini scan ZAP + 1 remédiation.
- 3. CI/CD minimale utile** : build/tests automatiques + build image Docker API + déploiement auto (ou script) sur DEV puis TEST via merge.
- 4. Prod** : procédure release (merge test → prod + tag), et procédure backup/restore validée (preuve), + doc utilisateur/admin + démo scénarisée.

Pré-requis (déjà acquis dans vos séances)

- Branches dev/test/prod + PR obligatoires, 3 VM DEV/TEST/PROD (ex 192.168.1.51/.52/.53), et “preuve de déploiement” nginx.
- DB MySQL Docker + Flyway + procédure mysqldump/restore (séance 5).
- DTO + Validation + gestion d’erreurs standardisées (séance 6).
- JWT/RBAC + front web + mobile opérationnels (séances 7-9-9bis).

Découpage 4h

10 min :

Kickoff + preuves attendues + “où on met quoi”

1h30 min :

Bloc A — Versions + Révisions + Audit + Durcissement + mini ZAP

1h30 min :

Bloc B — CI/CD + Docker image API + déploiement DEV → TEST (merge) + preuve sur TEST

50 min :

Bloc C — Release PROD + backup/restore validé + doc + démo finale

A - Kickoff “preuve E5 Bloc 1”

Pourquoi on fait ça maintenant (version “entreprise”)

- **Sans révisions/audit** : impossible d’expliquer un incident (“qui a modifié quoi ?”), impossible de revenir en arrière proprement.
- **Sans CI/CD** : vos merges dev→test→prod deviennent risqués (intégrité).
- **Sans backup/restore validé** : prod = roulette russe (disponibilité), et ça était déjà dans le registre des risques.

Preuves attendues

- GET /api/version retourne env + version (et on l’affiche sur VM TEST). (objectif 11)
- Modifier une page crée 1 révision consultable. (objectif 10)
- Audit log consultable (admin) : “DEV a fait UPDATE page X à telle heure”. (objectif 10)
- ZAP “baseline” : au moins 1 finding corrigé (souvent headers). (objectif 10)
- GitHub Actions : pipeline vert + déploiement TEST déclenché sur merge vers test. (objectif 11)
- Backup + restore : procédure exécutée et vérifiée (sur DEV ou TEST, pas PROD). (objectif 12 + séance 5)

B) Versions + Révisions + Audit + Durcissement + mini ZAP

Versions

C'est l'identification explicite d'un build déployé (numéro de version, hash Git, environnement DEV/TEST/PROD).

Objectif : ça sert à prouver "quelle version tourne où" et à diagnostiquer rapidement un incident ou une régression.

Révisions

C'est l'historique des modifications d'un contenu (ex : une page Wiki) : on conserve des "snapshots" ou des versions successives.

Objectif : traçabilité et possibilité de comprendre/justifier un changement (et parfois revenir en arrière).

Audit

C'est la journalisation des actions utilisateurs sur le système ("qui a fait quoi, quand, sur quelle ressource").

Objectif : responsabilité, conformité, analyse post-incident, et supervision côté admin.

Durcissement

C'est l'ensemble des mesures qui réduisent la surface d'attaque et les risques (validations strictes, en-têtes de sécurité, protection XSS, règles d'accès).

Objectif : On vise des gains rapides et mesurables sans "sur-architecturer".

Mini ZAP

C'est un scan de sécurité automatisé (OWASP ZAP baseline) pour détecter des faiblesses web évidentes (headers manquants, contenu mixte, etc.).

Objectif : On s'en sert pour obtenir un premier rapport, puis corriger au moins 1 point simple et re-scanner pour prouver l'amélioration.

OWASP = Open Worldwide Application Security Project.

C'est une organisation à but non lucratif qui produit des ressources gratuites sur la sécurité applicative (ex : le "Top 10" des risques web).

ZAP = Zed Attack Proxy (aussi appelé "OWASP ZAP").

C'est un outil OWASP de test de sécurité d'applications web (proxy/intercepteur + scanners) qui aide à détecter des failles (headers manquants, XSS, etc.).

B1) Endpoint /api/version (preuve “environnement + build”)

But : donner une preuve simple sur chaque VM (surtout TEST) après déploiement.

On ajoute ces propriétés pour exposer via /api/version une preuve fiable de l’environnement (DEV/TEST/PROD) et du build réellement déployé (hash/tag).

La syntaxe \${VAR:default} permet de prendre la valeur injectée sur les VM (variables d’environnement) et de retomber sur un défaut en local, sans configuration supplémentaire.

1. Dans le fichier : api/src/main/resources/application.properties , ajoute :

```
# Valeurs par défaut (surchargées par variables d’environnement sur VM)
mycellius.env=${MYCELLIUS_ENV:DEV}
mycellius.version=${MYCELLIUS_VERSION:local}
```

Puis configure IntelliJ :

File > Settings > File Encodings > Default encoding => UTF-8 + coche "Transparent native blablabla..." + relance un `mvn -f api/pom.xml spring-boot:run`

2. Nouveau contrôleur technique

Ce contrôleur sert à exposer un endpoint simple (ex. /api/version) qui renvoie l’environnement et la version du build en cours d’exécution.

Ça permet de vérifier immédiatement, après un déploiement sur une VM (TEST/PROD), que c’est bien la bonne version qui tourne, sans aller fouiller les logs.

> Crée le fichier : api/src/main/java/fr/mycellius/web/controller/MetaController.java :

```
package fr.mycellius.web.controller;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.Map;

@RestController
public class MetaController {

    @Value("${mycellius.env}")
    private String env;

    @Value("${mycellius.version}")
    private String version;

    @GetMapping("/api/version")
    public Map<String, String> version() {
        return Map.of(
            "env", env,
            "version", version
        );
    }
}
```

```
}  
}
```

Test local : GET http://localhost:8080/api/version → {"env":"DEV","version":"local"}

B2) Révisions: table + endpoint “list revisions for a page”

But : On stocke des “snapshots” (avant modification). Pas de diff, pas de rollback automatique, juste l’historique consultable (ça suffit pour l’objectif pédagogique).

1. Migration Flyway (V4)

On crée cette migration Flyway pour versionner l’évolution du schéma de BDD de façon reproductible (même setup sur DEV/TEST/PROD), sans “modifier à la main” la base.

*La table **wiki_page_revision** stocke un snapshot (titre+contenu) avant chaque modification afin d’avoir un historique consultable, et l’index sur page_id accélère la récupération des révisions d’une page.*

Si tu ne le fais pas, tu perds l’historique : après une modification, impossible de voir “ce qu’il y avait avant” et donc impossible de prouver qu’une révision a été créée (objectif de séance). En cas d’erreur ou de contenu saboté, tu n’as aucune trace exploitable pour comprendre ou reconstruire, et tu ne peux pas démontrer la traçabilité attendue (E5 : exploitation/suivi).

Exemple :

Dans l’appli, tu as une page Wiki (une “fiche”) qui s’appelle “Procédure Backup”. Son texte (content) contient une procédure importante, par exemple :

Étape 1 : arrêter l’API

Étape 2 : faire le dump MySQL

Étape 3 : restaurer la base

Un dev édite cette page via le CRUD (web ou mobile) et se trompe : il efface le contenu ou le remplace par un texte incorrect.

Avec les “révisions” activées, juste avant d’enregistrer la nouvelle version, l’API stocke automatiquement l’ancienne version (le snapshot) dans wiki_page_revision. Ensuite, tu peux consulter la liste des anciennes versions via /api/v1/pages/{id}/revisions et retrouver exactement le texte d’avant erreur.

Sans révisions, une fois sauvegardé, l’ancien contenu est perdu : tu n’as aucune copie consultable côté application.

Créer le fichier : `api/src/main/resources/db/migration/V6__revisions_and_audit.sql`

```
CREATE TABLE IF NOT EXISTS wiki_page_revision (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  page_id VARCHAR(64) NOT NULL,  
  title VARCHAR(255) NOT NULL,  
  content TEXT NOT NULL,  
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  created_by VARCHAR(64) NOT NULL  
);  
  
CREATE INDEX idx_revision_page_id ON wiki_page_revision(page_id);
```

(Rappel : vous êtes déjà sur Flyway et migrations versionnées depuis la séance 5.)

2. Entité + repository JPA

On crée une entité JPA pour “mapper” la table `wiki_page_revision` en objet Java manipulable par Spring (lecture/écriture sans SQL manuel).

“Mapper” veut dire “faire la correspondance” entre la table SQL et un objet Java.

Le repository associé (le “repository” est un composant Spring qui te sert de “porte d’entrée” vers la table, sans écrire de SQL) fournit les opérations standard (save, find...) et une requête simple pour lister rapidement les révisions d’une page (findByPageld...), ce qui alimente ensuite l’endpoint /revisions.

Si tu ne le fais pas, ton application ne saura pas accéder à la table `wiki_page_revision` via Spring Data JPA. Concrètement, tu ne pourras ni enregistrer automatiquement une révision, ni récupérer la liste des révisions via un endpoint, sauf à écrire du SQL “à la main” (JdbcTemplate) et gérer toi-même le mapping.

> Créer le fichier : `api/src/main/java/fr/mycellius/persistence/revision/WikiPageRevisionEntity.java`

```
package fr.mycellius.persistence.revision;  
  
import jakarta.persistence.*;  
import java.time.Instant;  
  
@Entity  
@Table(name = "wiki_page_revision")  
public class WikiPageRevisionEntity {
```

```

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

@Column(name = "page_id", nullable = false, length = 64)
private String pagelId;

@Column(nullable = false, length = 255)
private String title;

@Lob
@Column(nullable = false)
private String content;

@Column(name = "created_at", nullable = false)
private Instant createdAt;

@Column(name = "created_by", nullable = false, length = 64)
private String createdBy;

protected WikiPageRevisionEntity() {}

public WikiPageRevisionEntity(String pagelId, String title, String content, String
createdBy) {
    this.pagelId = pagelId;
    this.title = title;
    this.content = content;
    this.createdBy = createdBy;
    this.createdAt = Instant.now();
}

public Long getId() { return id; }
public String getPagelId() { return pagelId; }
public String getTitle() { return title; }
public String getContent() { return content; }
public Instant getCreatedAt() { return createdAt; }
public String getCreatedBy() { return createdBy; }
}

```

> Créer le fichier :

api/src/main/java/fr/mycellius/persistence/revision/WikiPageRevisionJpaRepository.java

Ce fichier définit l'interface "repository" Spring Data JPA pour la table wiki_page_revision.

Il te permet d'enregistrer et de lire des révisions sans écrire de requêtes SQL, et surtout d'avoir une méthode dédiée pour récupérer toutes les révisions d'une page par pageld (triées par date), utilisée ensuite par l'endpoint /revisions.

```
package fr.mycellius.persistence.revision;

import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface WikiPageRevisionJpaRepository extends
JpaRepository<WikiPageRevisionEntity, Long> {
    List<WikiPageRevisionEntity> findByPageldOrderByCreatedAtDesc(String pageld);
}
```

3. Service “création de révision avant update”

> Créer le fichier : api/src/main/java/fr/mycellius/service/RevisionService.java

Ce service sert à centraliser la logique métier “quand une page est modifiée, on enregistre d’abord un snapshot de l’état actuel”.

Il évite de dupliquer cette logique dans plusieurs contrôleurs et garantit que la création de révision se fait toujours de la même manière, en s’appuyant sur le repository.

```
package fr.mycellius.service;

import fr.mycellius.domain.WikiPage;
import fr.mycellius.persistence.revision.WikiPageRevisionEntity;
import fr.mycellius.persistence.revision.WikiPageRevisionJpaRepository;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class RevisionService {

    private final WikiPageRevisionJpaRepository repo;

    public RevisionService(WikiPageRevisionJpaRepository repo) {
        this.repo = repo;
    }

    public void createSnapshot(WikiPage currentPage, String actorUsername) {
```

```

        repo.save(new WikiPageRevisionEntity(
            currentPage.getId(),
            currentPage.getTitle(),
            currentPage.getContent(),
            actorUsername
        ));
    }

    public List<WikiPageRevisionEntity> list(String pageld) {
        return repo.findByPageldOrderByCreatedAtDesc(pageld);
    }
}

```

4. Brancher “createSnapshot” au bon endroit

On va ajouter un RevisionService au contrôleur, puis appeler createSnapshot juste avant updatePage(...) dans ta méthode @PutMapping.

cf la logique “controller → service” cadrée en séance 6.

On le branche juste avant l’update pour capturer l’ancienne version (l’état “avant modification”) : sinon tu enregistrerais la nouvelle version, ce qui n’a aucun intérêt pour une révision. On récupère donc la page actuelle, on la sauvegarde en révision, puis seulement ensuite on applique la mise à jour du CRUD.

Avant de modifier le PUT, on va d’abord :

- En haut du fichier, ajoute l’import : `import fr.mycellius.service.RevisionService;`
- Dans la classe, ajoute le champ : `private final RevisionService revisionService;`
- Modifie le constructeur pour l’injecteur :

```

public WikiPageController(WikiService wikiService, WikiPageDtoMapper mapper,
    RevisionService revisionService) {
    this.wikiService = wikiService;
    this.mapper = mapper;
    this.revisionService = revisionService;
}

```

Puis dans WikiPageController.java :

- Modifie la méthode PUT pour snapshotter “avant update”
- ajoutue avant la méthode update(...) :

```

// 1) récupérer l'état AVANT modification
WikiPage current = wikiService.getPageById(id);

```

```
// 2) enregistrer un snapshot (révision) de l'ancienne version
revisionService.createSnapshot(current, "SYSTEM"); // on mettra l'utilisateur réel
juste après
```

On va maintenant mettre l'utilisateur réel en place :

> Juste après tes imports existants, ajoute :

```
import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContextHolder;
```

> Dans ta méthode @PostMapping("/{id}"), tu as déjà :

```
WikiPage current = wikiService.getPageById(id);
revisionService.createSnapshot(current, "SYSTEM");
```

> remplace la ligne "SYSTEM" en insérant 2 lignes JUSTE APRÈS current, et AVANT createSnapshot :

```
// 2) récupérer le username depuis le JWT (Spring Security)
Authentication auth = SecurityContextHolder.getContext().getAuthentication();
String username = (auth != null) ? auth.getName() : "SYSTEM";

// 2bis) enregistrer un snapshot (révision) de l'ancienne version
revisionService.createSnapshot(current, username);
```

5. Endpoint révisions

> Créer le fichier : api/src/main/java/fr/mycellius/web/controller/RevisionController.java

On crée ce contrôleur pour exposer une route HTTP qui permet de consulter l'historique des versions d'une page (les snapshots enregistrés).

Concrètement, le front appelle /api/v1/pages/{id}/revisions et l'API renvoie la liste des révisions (date, auteur, contenu) sans modifier le CRUD existant.

Si tu ne le fais pas, les révisions existent peut-être en base (table + snapshots), mais elles restent invisibles pour l'extérieur : ni le front web/mobile ni un admin ne peuvent les consulter via l'API.

Donc tu ne peux pas démontrer l'objectif "endpoint révisions" et l'historique ne sert pratiquement à rien opérationnellement.

```
package fr.mycellius.web.controller;

import fr.mycellius.persistence.revision.WikiPageRevisionEntity;
import fr.mycellius.service.RevisionService;
import org.springframework.web.bind.annotation.*;
import java.util.List;
```

```

@RestController
@RequestMapping("/api/v1/pages")
public class RevisionController {

    private final RevisionService revisionService;

    public RevisionController(RevisionService revisionService) {
        this.revisionService = revisionService;
    }

    @GetMapping("/{id}/revisions")
    public List<WikiPageRevisionEntity> revisions(@PathVariable String id) {
        return revisionService.list(id);
    }
}

```

Test rapide :

- Faire 1 PUT sur une page existante (vous avez déjà CRUD web/mobile côté 9/9bis).
- Puis GET /api/v1/pages/{id}/revisions → au moins 1 entrée.

B3) Audit actions : table + log minimal sur POST/PUT/DELETE

But : “qui a fait quoi” (action + acteur + ressource + timestamp). On met en place l’audit pour tracer “qui a fait quoi, quand, sur quelle page” lors des opérations sensibles du CRUD (POST/PUT/DELETE). C’est une preuve de traçabilité utile en exploitation (diagnostic, responsabilité) et un attendu clair des objectifs de séance.

On ne fait pas une usine à gaz : 1 table, 1 service, 3 appels.

1. Migration Flyway

Crée une nouvelle migration :

Dans le fichier : api/src/main/resources/db/migration/V7__create_audit_log.sql

On ajoute cette migration Flyway pour créer la table audit_log de manière versionnée et reproductible sur toutes les VM (DEV/TEST/PROD), sans manipulation manuelle. Cette table stocke les preuves “qui a fait quoi” (action, acteur, ressource, date) à chaque POST/PUT/DELETE, ce qui permet la traçabilité et l’analyse en cas d’incident.

```

CREATE TABLE IF NOT EXISTS audit_log (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    action VARCHAR(32) NOT NULL,
    page_id VARCHAR(64) NULL,

```

```
actor VARCHAR(64) NOT NULL,  
created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE INDEX idx_audit_page_id ON audit_log(page_id);
```

N'oublie pas de redémarrer l'API pour que les migrations soient prises en compte !

2. JPA entity + repo

> Créer le fichier : `api/src/main/java/fr/mycellius/persistence/audit/AuditLogEntity.java`

On crée `AuditLogEntity` pour représenter la table `audit_log` en Java afin que Spring/Hibernate puisse enregistrer et lire les logs sans SQL manuel. Ça permet ensuite d'écrire une ligne d'audit à chaque action (CREATE/UPDATE/DELETE) et de les consulter facilement via un repository/endpoint.

```
package fr.mycellius.persistence.audit;  
  
import jakarta.persistence.*;  
import java.time.Instant;  
  
@Entity  
@Table(name = "audit_log")  
public class AuditLogEntity {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
  
    @Column(nullable = false, length = 32)  
    private String action;  
  
    @Column(name = "page_id", length = 64)  
    private String pageId;  
  
    @Column(nullable = false, length = 64)  
    private String actor;  
  
    @Column(name = "created_at", nullable = false)  
    private Instant createdAt;  
  
    protected AuditLogEntity() {}  
}
```

```

public AuditLogEntity(String action, String pageld, String actor) {
    this.action = action;
    this.pageld = pageld;
    this.actor = actor;
    this.createdAt = Instant.now();
}

public Long getId() { return id; }
public String getAction() { return action; }
public String getPageld() { return pageld; }
public String getActor() { return actor; }
public Instant getCreatedAt() { return createdAt; }
}

```

> Créer le fichier : `api/src/main/java/fr/mycellius/persistence/audit/AuditLogJpaRepository.java`

Ce repository sert d'accès simple à la table `audit_log` : `save(...)` pour enregistrer une action, et une méthode type `findByPageld...` pour lister rapidement les logs d'une page. Sans lui, tu devrais écrire du SQL à la main pour chaque lecture/écriture d'audit.

```

package fr.mycellius.persistence.audit;

import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface AuditLogJpaRepository extends JpaRepository<AuditLogEntity, Long> {
    List<AuditLogEntity> findByPageldOrderByCreatedAtDesc(String pageld);
}

```

3. Service audit

> Créer le fichier : `api/src/main/java/fr/mycellius/service/AuditService.java`

On crée `AuditService` pour centraliser la logique "écrire un log d'audit" (action, page, acteur, date) au même endroit, plutôt que de dupliquer du code dans chaque contrôleur. Ça garantit une trace cohérente et facilite l'évolution (ajouter un champ, filtrer, etc.) sans toucher tous les endpoints.

```

package fr.mycellius.service;

import fr.mycellius.persistence.audit.AuditLogEntity;
import fr.mycellius.persistence.audit.AuditLogJpaRepository;
import org.springframework.stereotype.Service;

import java.util.List;

@Service

```

```

public class AuditService {

    private final AuditLogJpaRepository repo;

    public AuditService(AuditLogJpaRepository repo) {
        this.repo = repo;
    }

    public void log(String action, String pageld, String actor) {
        repo.save(new AuditLogEntity(action, pageld, actor));
    }

    public List<AuditLogEntity> listByPageld(String pageld) {
        return repo.findByPageldOrderByCreatedAtDesc(pageld);
    }
}

```

4. Appels dans le contrôleur (POST/PUT/DELETE)

Dans le fichier : api/src/main/java/fr/mycellius/web/controller/WikiPageController.java

Pour l'audit, tu dois juste ajouter AuditService, centraliser la récupération du username, puis appeler auditService.log(...) dans POST/PUT/DELETE après succès.

On déclenche l'audit dans les contrôleurs parce que c'est là que l'action "métier" est confirmée comme réussie (création/modif/suppression OK). En loggant après succès, tu évites de tracer des opérations qui ont échoué et tu obtiens une preuve fiable "CREATE/UPDATE/DELETE" exploitable par l'admin.

a. Ajoute l'import

```
import fr.mycellius.service.AuditService;
```

b. Ajoute le champ + injecte dans le constructeur

```
private final AuditService auditService;
```

c. Puis modifie le constructeur :

```

public WikiPageController(WikiService wikiService, WikiPageDtoMapper mapper,
RevisionService revisionService, AuditService auditService) {
    this.wikiService = wikiService;
    this.mapper = mapper;
    this.revisionService = revisionService;
    this.auditService = auditService;
}

```

d. Ajoute une méthode utilitaire pour le username (et supprime le bloc "Authentication auth..." dans le PUT)

Ajoute dans la classe :

```
private String currentUsername() {
    Authentication auth = SecurityContextHolder.getContext().getAuthentication();
    return (auth != null) ? auth.getName() : "SYSTEM";
}
```

e. Ajoute les logs d'audit dans POST/PUT/DELETE

POST (juste après createPage(...)) :

```
WikiPage created = wikiService.createPage(mapper.toDomain(request));
auditService.log("CREATE", created.getId(), currentUsername());
```

PUT (remplace ton bloc username + utilise currentUsername(), et log après update) :

```
WikiPage current = wikiService.getPageById(id);
revisionService.createSnapshot(current, currentUsername());

WikiPage updated = wikiService.updatePage(id, mapper.toDomain(request));
auditService.log("UPDATE", id, currentUsername());
return mapper.toResponse(updated);
```

DELETE (après deletePage) :

```
wikiService.deletePage(id);
auditService.log("DELETE", id, currentUsername());
```

Voici le code que tu dois avoir dans ce fichier après modification :

```
package fr.mycellius.web.controller;

import fr.mycellius.domain.WikiPage;
import fr.mycellius.service.WikiService;
import fr.mycellius.service.RevisionService;
import fr.mycellius.service.AuditService;
import fr.mycellius.web.dto.CreateWikiPageRequest;
import fr.mycellius.web.dto.WikiPageResponse;
import fr.mycellius.web.mapper.WikiPageDtoMapper;
import jakarta.validation.Valid;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/v1/pages")
```

```

public class WikiPageController {

    private final WikiService wikiService;
    private final WikiPageDtoMapper mapper;
    private final RevisionService revisionService;
    private final AuditService auditService;

    public WikiPageController(WikiService wikiService,
                              WikiPageDtoMapper mapper,
                              RevisionService revisionService,
                              AuditService auditService) {
        this.wikiService = wikiService;
        this.mapper = mapper;
        this.revisionService = revisionService;
        this.auditService = auditService;
    }

    private String currentUsername() {
        Authentication auth = SecurityContextHolder.getContext().getAuthentication();
        return (auth != null) ? auth.getName() : "SYSTEM";
    }

    @PostMapping
    public ResponseEntity<WikiPageResponse> createPage(@Valid @RequestBody
CreateWikiPageRequest request) {
        WikiPage created = wikiService.createPage(mapper.toDomain(request));
        auditService.log("CREATE", created.getId(), currentUsername());
        return
ResponseEntity.status(HttpStatus.CREATED).body(mapper.toResponse(created));
    }

    @GetMapping
    public Page<WikiPageResponse> list(@RequestParam(defaultValue = "0") int page,
                                       @RequestParam(defaultValue = "10") int size) {
        PageRequest pageable = PageRequest.of(page, size);
        return wikiService.listPages(pageable).map(mapper::toResponse);
    }

    @GetMapping("/{id}")
    public WikiPageResponse getPageById(@PathVariable String id) {
        return mapper.toResponse(wikiService.getPageById(id));
    }

    @GetMapping("/search")
    public Page<WikiPageResponse> searchByTitle(@RequestParam("title") String
fragment,
                                               @RequestParam(defaultValue = "0") int page,

```

```

        @RequestParam(defaultValue = "10") int size) {
    PageRequest pageable = PageRequest.of(page, size);
    return wikiService.searchByTitle(fragment, pageable).map(mapper::toResponse);
}

@DeleteMapping("/{id}")
@ResponseStatus(HttpStatus.NO_CONTENT)
public void delete(@PathVariable String id) {
    wikiService.deletePage(id);
    auditService.log("DELETE", id, currentUsername());
}

@PutMapping("/{id}")
public WikiPageResponse update(@PathVariable String id,
    @Valid @RequestBody CreateWikiPageRequest request) {

    WikiPage current = wikiService.getPageById(id);
    revisionService.createSnapshot(current, currentUsername());

    WikiPage updated = wikiService.updatePage(id, mapper.toDomain(request));
    auditService.log("UPDATE", id, currentUsername());

    return mapper.toResponse(updated);
}
}

```

On passe au test de l'audit (vérifier que audit log se remplit) :

1. Vérifie que les tables existent dans MySQL puis exécute ces 2 commandes (dans ton client SQL ou via Docker) :

```

USE mycellius;
SHOW TABLES LIKE 'audit_log';

USE mycellius;
SHOW TABLES LIKE 'wiki_page_revision';

```

Résultat attendu : chaque commande doit renvoyer 1 ligne (le nom de la table).

2. On génère 2 actions et on vérifie en DB.

- > Fais une création (POST) d'une page via Swagger ou ton front (en DEV ou ADMIN). Récupère l'ID créé.
- > Fais une modification (PUT) de cette même page (change le title ou content).
- > Vérifie en SQL (copie/colle le résultat) :

```
USE mycellius;
SELECT COUNT(*) AS nb_revisions FROM wiki_page_revision;
SELECT action, page_id, actor FROM audit_log ORDER BY id DESC LIMIT 5;
```

Attendu :

- > nb_revisions >= 1 après le PUT
- > dans audit_log, tu dois voir au moins CREATE puis UPDATE (avec la colonne action de MySQLWorkbench).

3. On va vérifier que le snapshot est bien créé après un PUT.

Exécute en SQL :

```
USE mycellius;

-- Dernière révision créée (doit apparaître juste après ton PUT)
SELECT id, page_id, created_by, created_at, title
FROM wiki_page_revision
ORDER BY id DESC
LIMIT 1;
```

Attendu : au moins 1 ligne, avec page_id = la page modifiée et created_by = ton user.

4. On va vérifier que la révision correspond bien à "l'ancienne version" (avant le PUT).

- > Prends l'id de la page que tu as modifiée (ex PAGE-001).
- > Exécute ce bloc SQL (copie/colle) :

```
USE mycellius;

-- Version actuelle (après PUT)
SELECT id, title, content
FROM wiki_page
WHERE id = 'PAGE-001';

-- Dernière révision (snapshot avant PUT)
SELECT title, content, created_by, created_at
FROM wiki_page_revision
WHERE page_id = 'PAGE-001'
```

```
ORDER BY id DESC
LIMIT 1;
```

Attendu (onglets en-dessous de la section Output):

- > wiki_page = contenu modifié,
- > wiki_page_revision = contenu d'avant modification.

5. Endpoint audit (ADMIN only)

On crée cet endpoint pour permettre à un ADMIN de consulter les traces d'audit d'une page (qui a créé/modifié/supprimé, quand). Le rendre "ADMIN only" évite d'exposer des informations sensibles (identités, actions internes) aux autres rôles.

Avant d'ajouter la règle RBAC, créer le fichier :

api/src/main/java/fr/mycellius/web/controller/AuditController.java

```
package fr.mycellius.web.controller;

import fr.mycellius.persistence.audit.AuditLogEntity;
import fr.mycellius.service.AuditService;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/v1/audit")
public class AuditController {

    private final AuditService auditService;

    public AuditController(AuditService auditService) {
        this.auditService = auditService;
    }

    @GetMapping("/pages/{id}")
    public List<AuditLogEntity> auditPage(@PathVariable String id) {
        return auditService.listByPageId(id);
    }
}
```

6. On va ajouter la règle RBAC "ADMIN only" pour l'audit endpoint.

Dans ton **SecurityConfig** actuel, tu n'as pas la règle RBAC pour l'audit.
Tu as uniquement les règles `/api/v1/pages/**`, puis tu termines par `.anyRequest().authenticated()`.

Donc, ajoute avant `.anyRequest().authenticated()` :

```
// RBAC Audit (ADMIN only)
.requestMatchers(HttpMethod.GET, "/api/v1/audit/**")
.hasRole("ADMIN")
```

Extrait complet (à coller au bon endroit) :

```
.requestMatchers(HttpMethod.DELETE, "/api/v1/pages/**")
.hasAnyRole("DEV", "ADMIN")

// RBAC Audit (ADMIN only)
.requestMatchers(HttpMethod.GET, "/api/v1/audit/**")
.hasRole("ADMIN")

.anyRequest().authenticated()
```

On autorise `/error` en ajoutant cette ligne dans `authorizeHttpRequests`, juste après `login/health`) :

```
.requestMatchers("/error").permitAll()
```

Remplace ton `accessDeniedHandler` pour ne pas déclencher un dispatch `/error` :

```
.accessDeniedHandler((req, res, ex) -> res.sendError(HttpStatus.FORBIDDEN.value()))
```

Remplace ton `accessDeniedHandler` pour ne pas déclencher un dispatch `/error` :

```
.accessDeniedHandler((req, res, ex) -> res.setStatus(HttpStatus.FORBIDDEN.value()))
```

Teste rapidement sur postman :

- > Connecte-toi en DEV → appelle GET `/api/v1/audit/pages/PAGE-001` → attendu 403.
- > Connecte-toi en ADMIN → même appel → attendu 200 + liste.

=> **Pour rappel :**

audit_log sert à garder une trace fiable des actions sensibles dans l'application : "qui a créé/modifié/supprimé quoi, et quand".

Concrètement, ça sert à 3 choses très utiles en exploitation :

Diagnostic d'incident

Une page a été modifiée/supprimée "mystérieusement" : l'audit te dit quel compte l'a fait et à quel moment. Sans ça, tu es à l'aveugle.

Responsabilité / conformité

En contexte entreprise, tu dois souvent pouvoir prouver les actions (traçabilité). C'est exactement ce que demande le référentiel côté exploitation/sécurisation (E5).

Sécurité

Si un compte est compromis ou qu'un stagiaire fait une erreur, tu peux détecter une activité anormale (beaucoup de DELETE/UPDATE) et réagir.

Et c'est complémentaire des "révisions" :

> révisions = "qu'est-ce qui a changé (contenu avant) ?"

> audit_log = "qui a déclenché le changement (action, acteur, date) ?"

B4) Durcissement minimal + mini ZAP (remédiation “rapide et visible”)

But :

Durcissement = 3 gestes qui “tombent juste” pédagogiquement :

- validations,
- headers,
- XSS Markdown.

1. Validations : renforcer ce qui est “risque direct”

Vous avez déjà Jakarta Validation (séance 6).

On renforce les validations pour bloquer dès l'entrée des données invalides ou dangereuses (contenu vide, trop long, format d'ID non conforme), avant qu'elles n'atteignent la base ou le front. Ça réduit les risques d'erreurs, de surcharge (payload énorme) et de comportements inattendus côté UI/API. Et c'est une mesure “durcissement” simple, mesurable, cohérente avec Jakarta Validation déjà en place.

Dans les DTO de requête côté API, ceux qui reçoivent les données du front (les classes utilisées avec @RequestBody + @Valid).

Dans ton projet, c'est dans le fichier:

api/src/main/java/fr/mycellius/web/dto/CreateWikiPageRequest.java

Pour les id, remplace :

```
@Pattern(regexp = "PAGE-\\d{3}", message = "Format d'id attendu : PAGE-001")
```

Par :

```
@Pattern(regexp = "^PAGE-\\d{3}$", message = "Format d'id attendu : PAGE-001")
```

Pour les titres, rajoute :

```
@Valid  
List<TagRequest> tags
```

Par :

```
@Size(max = 20, message = "Max 20 tags")  
@Valid  
List<TagRequest> tags
```

=> Ça évite qu'un client envoie 5000 tags et te surcharge l'API/DB.

2. Security headers (souvent le finding le plus simple à corriger)

Dans votre SecurityConfig (Spring Security), ajoute ce bloc `.headers(...)` dans ta chaîne http, juste après `.formLogin(f -> f.disable())` (avant `.authorizeHttpRequests(...)`) :

```
.headers(headers -> headers
    .frameOptions(frame -> frame.deny())
    .contentTypeOptions(Customizer.withDefaults())
    .referrerPolicy(ref -> ref.policy(
        org.springframework.security.web.header.writers.ReferrerPolicyHeaderWriter.ReferrerPolicy
        .NO_REFERRER
    ))
)
```

Puis redémarre l'API et vérifie en 1 commande : `curl.exe -I http://localhost:8080/api/health`

Attendu dans les headers : X-Frame-Options: DENY et X-Content-Type-Options: nosniff et Referrer-Policy: no-referrer.

```
PS C:\Users\clakg\mycellius> curl.exe -I http://localhost:8080/api/health
HTTP/1.1 200
Vary: Origin
Vary: Access-Control-Request-Method
Vary: Access-Control-Request-Headers
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: DENY
Referrer-Policy: no-referrer
Content-Type: application/json
Date: Mon, 23 Feb 2026 11:09:36 GMT
```

Ça suffit souvent à faire baisser un rapport ZAP "init". (Et c'est cohérent avec /health + actuator mentionnés depuis séance 4.)

3. XSS Markdown

Installer marked + dompurify et utiliser SafeMarkdown sert à afficher le contenu Markdown en HTML tout en supprimant ce qui pourrait exécuter du code (XSS).

Concrètement, la règle c'est :

Markdown → HTML (marked) → nettoyage sécurité (DOMPurify) → rendu React.

Si tu ne le fais pas et que tu rends du HTML/Markdown "brut", un utilisateur peut injecter du contenu malveillant (script, liens piégés) qui s'exécute chez les autres utilisateurs.

- Dans web/ :

```
cd web
npm i marked dompurify
```

- Puis créer le composant (ex : web/src/components/SafeMarkdown.jsx) :

```
import DOMPurify from "dompurify";
import { marked } from "marked";

export default function SafeMarkdown({ markdown }) {
  const html = DOMPurify.sanitize(marked.parse(markdown ?? ""));
  return <div dangerouslySetInnerHTML={{ __html: html }} />;
}
```

- L'utiliser là où tu rends le contenu (page détail) :

Dans ton composant de détail (ex PageDetailPage.jsx), remplace l'affichage brut :

```
<pre style={{ whiteSpace: "pre-wrap" }}>{page.content}</pre>
```

par l'affichage brut en rajoutant l'import évidemment:

```
import SafeMarkdown from "../components/SafeMarkdown";
// ...
<div style={{ whiteSpace: "normal" }}>
  <SafeMarkdown markdown={page.content} />
</div>
```

4. Mini scan ZAP (mode "1 commande")

Objectif : sortir un rapport + corriger 1 finding (headers), pas faire un pentest complet.

Le mini scan ZAP sert à obtenir rapidement un rapport automatique sur des faiblesses web "évidentes" (headers manquants, mauvaises pratiques, exposition basique) et à prouver une remédiation simple et mesurable.

Ça te donne une preuve de démarche sécurité (audit → correction) sans faire un pentest complet.

Si tu ne le fais pas, tu n'as ni diagnostic outillé ni preuve objective d'amélioration, et des failles triviales peuvent rester en prod sans que tu t'en rendes compte.

On lance le scan avec Docker sur la machine depuis la racine C:\Users\clakg\mycellius (ou n'importe quel dossier), l'intérêt c'est surtout que le rapport **zap_report.html** soit créé à un endroit que tu retrouves.

> Pull d'une image maintenue

```
docker pull ghcr.io/zaproxy/zaproxy:stable
```

/zaproxzy/zaproxy:stable est l'image Docker officielle (hébergée sur GitHub Container Registry) qui contient OWASP ZAP prêt à l'emploi. Elle te permet de lancer ZAP (dont zap-baseline.py) sans installer ZAP sur ton PC, juste via Docker.

> Lancer le baseline scan + générer un rapport récupérable sur ton disque

(important : -v \${PWD}:/zap/wrk sinon le rapport disparaît avec --rm)

```
docker run --rm -t -v ${PWD}:/zap/wrk ghcr.io/zaproxy/zaproxy:stable zap-baseline.py -t  
http://host.docker.internal:8080 -r zap_report.html
```

> Ouvrir le rapport

Tu trouveras zap_report.html dans C:\Users\clakg\mycellius.

Je clique-droit sur le fichier dans le Projet mycellius et je l'ouvre dans **Open in > Explorer** (cf capture page suivante)

=> Sans montage de volume, le rapport peut être perdu (créé dans le conteneur puis supprimé).

Sur le rapport HTML, tu dois vérifier 3 choses (rapide) :

- "Summary of Alerts"

=> Regarde s'il y a des alertes en High/Medium/Low. Chez moi : 0/0/0 → c'est bon.

- La liste "Alerts" (en bas de la page)

Note le nom des alertes restantes. Chez moi : Non-Storable Content (Informational) uniquement.

- "Alert Detail" de cette alerte

Vérifie que c'est bien un point mineur (cache-control/no-store) et pas un truc sécurité critique (XSS, headers manquants, cookies). Là, c'est informatif, donc objectif "mini scan" validé.

Conclusion :

Le scan ci-dessous est “propre” et les headers manquants ne ressortent plus.

Si tu dois livrer une preuve, garde ce rapport + une capture du “Summary of Alerts”.



ZAP Scanning Report

Site: <http://host.docker.internal:8080>

Generated on Mon, 23 Feb 2026 16:06:08

ZAP Version: 2.17.0

ZAP by [Checkmarx](#)

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	0
Low	0
Informational	1
False Positives	0

Insights

Level	Reason	Site	Description	Statistic
Info	Informational	http://host.docker.internal:8080	Percentage of responses with status code 4xx	100 %
Info	Informational	http://host.docker.internal:8080	Percentage of endpoints with method GET	100 %
Info	Informational	http://host.docker.internal:8080	Count of total endpoints	2
Info	Informational	http://host.docker.internal:8080	Percentage of slow responses	25 %

Summary of Sequences

For each step: result (Pass/Fail) - risk (of highest alert(s) for the step, if any).

Alerts

Name	Risk Level	Number of Instances
Non-Storable Content	Informational	3

Alert Detail

Informational	Non-Storable Content
Description	The response contents are not storable by caching components such as proxy servers. If the response does not contain sensitive, personal or user-specific information, it may benefit from being stored and cached, to improve performance.
URL	http://host.docker.internal:8080
Node Name	http://host.docker.internal:8080
Method	GET
Parameter	
Attack	
Evidence	no-store
Other Info	

C) CI/CD + Docker image API + déploiement DEV→TEST (merge) + preuve version sur TEST

C1) Dockerfile API (image “propre”)

But : donner une preuve simple sur chaque VM (surtout TEST) après déploiement.

Cela va permettre un build reproductible de l'API dans Docker (même résultat en local/CI/VM) et embarquer les tests dans l'étape build.

Sans : tes VM dépendent d'un build local ou d'un environnement “à la main”, et la CI ne peut pas prouver facilement qu'une image est buildable.

1. Créer le fichier : api/Dockerfile (à la racine du dossier api/) et ajoute :

```
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn -q -DskipTests=false test package
```

```
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

2. Test local (preuve que le Dockerfile marche) depuis la racine du projet :

```
docker build -t mycellius-api:local ./api
```

=> **Attendu :** build OK (pas d'erreur), et les tests passent pendant le build.

C2) GitHub Actions : build/tests + build image

Vous aviez une CI “garde-fou” dès séance 1 et l’idée de la compléter ensuite.

On veut que à chaque push/merge, GitHub prouve automatiquement que l’API compile, que les tests passent, et que l’image Docker se construit.

Sans : tu peux merger du code cassé et t’en rendre compte seulement au déploiement.

But : Vous aviez une CI “garde-fou” dès séance 1 et l’idée de la compléter ensuite.

1. Dans le fichier : _.github/workflows/ci.yml , remplace par :

```
name: CI

on:
  push:
    branches: [ "dev", "test", "prod" ]
  pull_request:
    branches: [ "test", "prod" ]

jobs:
  api-tests:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Java 21
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: "21"
          cache: maven

      - name: Build and run tests for API
        run: mvn -f api/pom.xml -B test

  api-docker-build:
    runs-on: ubuntu-latest
    needs: [ api-tests ]
    steps:
      - uses: actions/checkout@v4

      - name: Docker build (API)
        run: docker build -t mycellius-api:${{ github.sha }} ./api
```

Avant de faire un commit + push, on va créer un .gitignore à la racine du projet :

On en crée un parce qu'on a créé des fichiers qui vont générer des bugs lors du commit push.

On ne souhaite pas commit / push pour ne pas listés **mysql.sock, infra/db/data/** et **api/target/**

> Crée le **.gitignore** à la racine (dans ~/mycellius)

> dans le terminal, colle le code suivant :

```
cat > .gitignore <<'EOF'
# IntelliJ
.idea/
*.iml

# Maven
**/target/

# Node
web/node_modules/
web/dist/

# Docker/MySQL volumes (NE PAS versionner)
infra/db/data/
infra/stack/data/
*.sock

# Rapports ZAP (optionnel)
zap_report*.html
EOF
```

On va maintenant faire un commit + push, je vous laisse faire car vous commencer à connaître.

Vérification attendue

Dans GitHub Actions, le workflow doit exécuter 2 jobs : api-tests puis api-docker-build, et le run doit être vert.

Pour rappel :

Dans GitHub, base = la branche qui va recevoir tes changements (la branche “cible”). compare = ta branche de travail (celle où tu as développé).

Dans ton workflow dev → test → prod :

PR dev → test : base = test | compare = dev

PR test → prod (mise en prod) : base = prod | compare = test

=> Donc là, pour promouvoir ton travail actuel : base test, compare dev.

All checks have passed

2 successful checks

CI / api-tests (pull_request) Successful in 27s

CI / api-tests (push) Successful in 21s

No conflicts with base branch

Merging can be performed automatically.

Merge pull request

You can also merge this with the cor

C3) Déploiement auto “ou script” sur VM DEV puis TEST

Vous avez déjà les VM et le workflow dev→test→prod (séance 1).

But : Vous aviez une CI “garde-fou” dès séance 1 et l’idée est de la compléter ensuite.
La voie la plus fiable est : SSH + script sur VM qui fait git pull + docker compose up.

Ce qu'on a fait et qui est prêt :

- => sur les VMs DEV/TEST/PROD, on a /var/www/html
- => on n'a que index.html (pas le repo)
- => Docker est installé sur les 3 VMs (version affichée).

Connectez-vous en SSH à chaque machine pour vérifier ces 3 points.

```

clarisse@mycellius-dev: /var/www/html
Activez ESM Apps pour recevoir des futures mi
s.
Visitez https://ubuntu.com/esm ou exécutez :

The list of available updates is more than a
To check for new updates run: sudo apt update

Last login: Mon Jan 12 10:34:01 2026 from 192.168.
clarisse@mycellius-dev:~$ cd /
clarisse@mycellius-dev:/$ cd /var/www/html/
clarisse@mycellius-dev:/var/www/html$ ls
index.html
clarisse@mycellius-dev:/var/www/html$ cat index.html
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Mycellius - DEV</title>
  </head>
  <body>
    <p><b>ENV :</b> DEV</p>
  </body>
</html>
clarisse@mycellius-dev:/var/www/html$ pwd
/var/www/html
clarisse@mycellius-dev:/var/www/html$ docker --version
Docker version 29.1.2, build 890dcca
clarisse@mycellius-dev:/var/www/html$

clarisse@mycellius-test: /var/www/html
The list of available updates is more than a week
To check for new updates run: sudo apt update

Last login: Sat Dec 6 03:00:53 2025 from 192.168.
clarisse@mycellius-test:~$ cd /
clarisse@mycellius-test:/$ cd /var/www/html/
-bash: /var/www/html/: Is a directory
clarisse@mycellius-test:/var/www/html$ cd /var/www/html/
clarisse@mycellius-test:/var/www/html$ ls
index.html
clarisse@mycellius-test:/var/www/html$ cat index.html
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Mycellius - TEST</title>
  </head>
  <body>
    <p><b>ENV :</b> TEST</p>
    <p><b>branch dev -> branche test</p>
  </body>
</html>
clarisse@mycellius-test:/var/www/html$ pwd
/var/www/html
clarisse@mycellius-test:/var/www/html$ docker --version
Docker version 29.1.2, build 890dcca
clarisse@mycellius-test:/var/www/html$

Selection clarisse@mycellius-prod: /var/www/html
/home/clarisse
clarisse@mycellius-prod:~$ ls
bin boot dev home lib64 lost+found mnt proc
run sbin usr-is-merged srv sys usr
bin usr-is-merged cdrom etc lib lib usr-is-merged media opt root
sbin snap swap.img var
clarisse@mycellius-prod:/$ cd /var/www/html/
clarisse@mycellius-prod:/var/www/html$ ls
index.html
clarisse@mycellius-prod:/var/www/html$ cat index.html
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Mycellius - PROD</title>
  </head>
  <body>
    <p><b>ENV :</b> PROD</p>
    <p><b>branch dev -> branche prod</p>
  </body>
</html>
clarisse@mycellius-prod:/var/www/html$ pwd
/var/www/html
clarisse@mycellius-prod:/var/www/html$ docker --version
Docker version 29.1.2, build 890dcca
clarisse@mycellius-prod:/var/www/html$

```

C.3.1 - Cloner le repo sur chaque VM (sans casser ton index.html)

Ce qu'on va faire :

1 - Crée un sous-dossier pour le repo dans chaque VM (dev / test / prod)

```
cd /var/www/html
sudo mkdir -p mycellius
sudo chown -R $USER:$USER /var/www/html/mycellius
```

2 - Clone ton repo dedans (toujours dans chaque VM (dev / test / prod))

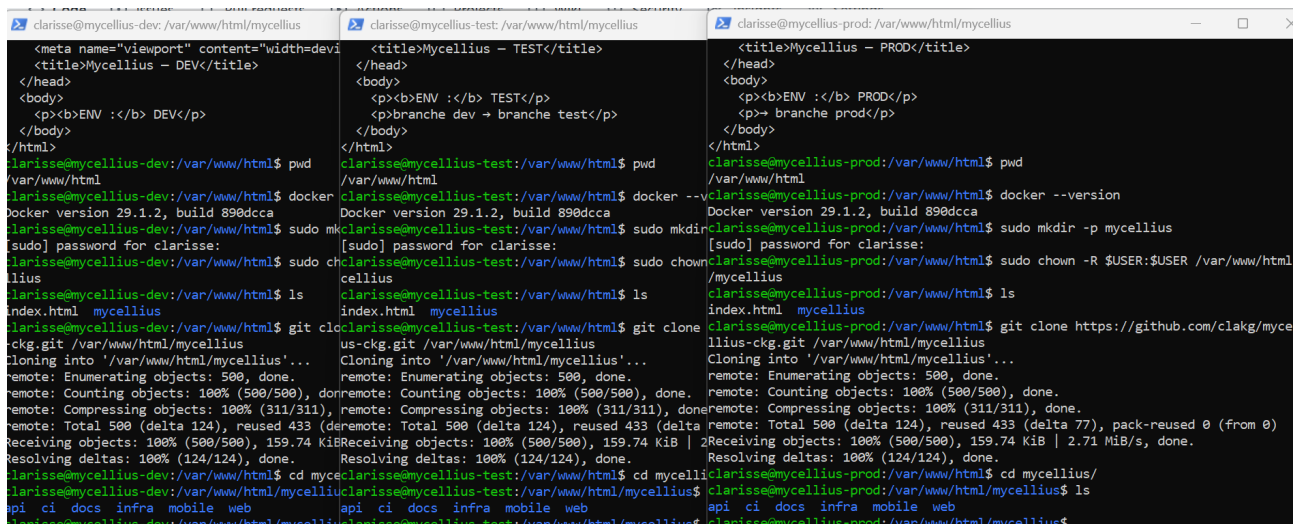
```
git clone <URL_GITHUB_DU_REPO> /var/www/html/mycellius

pour moi :
git clone https://github.com/clakg/mycellius-ckg.git /var/www/html/mycellius
```

3 - Vérifie que tu vois bien api/, infra/, etc. (dans chaque VM (dev / test / prod))

```
cd /var/www/html/mycellius
ls
```

=> Important : on ne touche pas à /var/www/html/index.html, donc ta preuve Nginx reste intacte.



The image shows three terminal windows side-by-side, each representing a different VM environment: dev, test, and prod. Each window shows the same sequence of commands being executed to set up the mycellius repository. The commands include navigating to the web directory, creating the mycellius subdirectory, changing ownership, cloning the repository from GitHub, and listing the contents to verify the structure. The output of the 'ls' command shows the presence of directories like 'api', 'ci', 'docs', 'infra', 'mobile', and 'web'.

```
clarisse@mycellius-dev: /var/www/html/mycellius$ cd /var/www/html
clarisse@mycellius-dev: /var/www/html$ sudo mkdir -p mycellius
clarisse@mycellius-dev: /var/www/html$ sudo chown -R $USER:$USER /var/www/html/mycellius
clarisse@mycellius-dev: /var/www/html$ git clone https://github.com/clakg/mycellius-ckg.git /var/www/html/mycellius
Cloning into '/var/www/html/mycellius'...
remote: Enumerating objects: 500, done.
remote: Counting objects: 100% (500/500), done.
remote: Compressing objects: 100% (311/311), done.
remote: Total 500 (delta 124), reused 433 (delta 77), pack-reused 0 (from 0)
Receiving objects: 100% (500/500), 159.74 KiB | 2.71 MiB/s, done.
Resolving deltas: 100% (124/124), done.
clarisse@mycellius-dev: /var/www/html/mycellius$ ls
api  ci  docs  infra  mobile  web

clarisse@mycellius-test: /var/www/html/mycellius$ cd /var/www/html
clarisse@mycellius-test: /var/www/html$ sudo mkdir -p mycellius
clarisse@mycellius-test: /var/www/html$ sudo chown -R $USER:$USER /var/www/html/mycellius
clarisse@mycellius-test: /var/www/html$ git clone https://github.com/clakg/mycellius-ckg.git /var/www/html/mycellius
Cloning into '/var/www/html/mycellius'...
remote: Enumerating objects: 500, done.
remote: Counting objects: 100% (500/500), done.
remote: Compressing objects: 100% (311/311), done.
remote: Total 500 (delta 124), reused 433 (delta 77), pack-reused 0 (from 0)
Receiving objects: 100% (500/500), 159.74 KiB | 2.71 MiB/s, done.
Resolving deltas: 100% (124/124), done.
clarisse@mycellius-test: /var/www/html/mycellius$ ls
api  ci  docs  infra  mobile  web

clarisse@mycellius-prod: /var/www/html/mycellius$ cd /var/www/html
clarisse@mycellius-prod: /var/www/html$ sudo mkdir -p mycellius
clarisse@mycellius-prod: /var/www/html$ sudo chown -R $USER:$USER /var/www/html/mycellius
clarisse@mycellius-prod: /var/www/html$ git clone https://github.com/clakg/mycellius-ckg.git /var/www/html/mycellius
Cloning into '/var/www/html/mycellius'...
remote: Enumerating objects: 500, done.
remote: Counting objects: 100% (500/500), done.
remote: Compressing objects: 100% (311/311), done.
remote: Total 500 (delta 124), reused 433 (delta 77), pack-reused 0 (from 0)
Receiving objects: 100% (500/500), 159.74 KiB | 2.71 MiB/s, done.
Resolving deltas: 100% (124/124), done.
clarisse@mycellius-prod: /var/www/html/mycellius$ ls
api  ci  docs  infra  mobile  web
```

C3.2 — Mettre en place la stack Docker sur les VMs (docker-compose)

But : c'est ce qui lance réellement MySQL + l'API de façon reproductible sur chaque VM.

Sans : le script de déploiement ne peut pas démarrer l'app (pas de stack standard).

À faire maintenant (sur les VMs, mais c'est du code versionné donc identique partout) :

1. Script de déploiement sur VM

Créer un script de déploiement permet de standardiser le déploiement (mêmes actions à chaque fois) et permettre à GitHub Actions de déclencher un déploiement via SSH sans bricolage manuel. Impact si tu le fais : un "push/merge" peut déclencher un déploiement reproductible (preuve DEV/TEST à jour).

Impact si tu ne le fais pas : déploiements manuels peut engendrer des erreurs, des oublis, et aucune preuve propre que "cette version a été déployée".

Créer et ajoute dans le fichier : infra/deploy/deploy.sh (exécuté sur la VM)

```
#!/usr/bin/env bash
set -euo pipefail

BRANCH="${1:-dev}"
ENV_NAME="${2:-DEV}"

cd /var/www/html/mycellius

git fetch origin
git checkout "$BRANCH"
git reset --hard
git clean -fd
git pull origin "$BRANCH"

export MYCELLIUS_ENV="$ENV_NAME"
export MYCELLIUS_VERSION="$(git rev-parse --short HEAD)"

docker compose -f infra/stack/docker-compose.yml up -d --build
echo "Deployed $ENV_NAME / $MYCELLIUS_VERSION"
```

Pensez au chmod et vérifie que Docker Compose est OK :

```
chmod +x infra/deploy/deploy.sh
```

Puis :

```
docker compose version
```

2. docker-compose “stack” (sur VM) : DB + API ensemble

Ajoute dans le fichier : infra/stack/docker-compose.yml

```
version: "3.9"
services:
  mysql:
    image: mysql:8.0
    container_name: mycellius-mysql
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: rootpassword
      MYSQL_DATABASE: mycellius
      MYSQL_USER: mycellius
      MYSQL_PASSWORD: mycellius
    volumes:
      - ./data:/var/lib/mysql

  api:
    build: ../../api
    container_name: mycellius-api
    restart: unless-stopped
    environment:
      SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/mycellius?
      useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
      SPRING_DATASOURCE_USERNAME: mycellius
      SPRING_DATASOURCE_PASSWORD: mycellius
      MYCELLIUS_ENV: ${MYCELLIUS_ENV}
      MYCELLIUS_VERSION: ${MYCELLIUS_VERSION}
    ports:
      - "8080:8080"
    depends_on:
      - mysql
```

(Le bloc MySQL est cohérent avec ce que vous aviez déjà en séance 5, juste “intégré” au stack VM.)

3. GitHub Actions : job deploy DEV si push sur dev, deploy TEST si push sur test

Ajoute dans **.github/workflows/ci.yml**, au même niveau que **api-tests** et **api-docker-build** (c'est-à-dire sous **jobs**).

Concrètement, tu colles **deploy-dev et **deploy-test** après **api-docker-build** (ou après **api-tests**, mais mieux après **docker-build** si tu veux une chaîne logique) : (fichier complet page suivante)**

```
deploy-dev:
  runs-on: [ self-hosted, dev-runner ]
  needs: [ api-tests, api-docker-build ]
  if: github.event_name == 'push' && github.ref == 'refs/heads/dev'
  steps:
    - name: Ensure deploy script is executable
      run: |
        cd /var/www/html/mycellius
        chmod +x infra/deploy/deploy.sh
    - name: Deploy DEV
      run: |
        cd /var/www/html/mycellius
        ./infra/deploy/deploy.sh dev DEV

deploy-test:
  runs-on: [ self-hosted, test-runner ]
  needs: [ api-tests, api-docker-build ]
  if: github.event_name == 'push' && github.ref == 'refs/heads/test'
  steps:
    - name: Ensure deploy script is executable
      run: |
        cd /var/www/html/mycellius
        chmod +x infra/deploy/deploy.sh
    - name: Deploy TEST
      run: |
        cd /var/www/html/mycellius
        ./infra/deploy/deploy.sh test TEST
```

Le fichier final ci.yml :

```
name: CI

on:
  push:
    branches: [ "dev", "test", "prod" ]
  pull_request:
    branches: [ "test", "prod" ]

jobs:
  api-tests:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Java 21
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: "21"
          cache: maven

      - name: Build and run tests for API
        run: mvn -f api/pom.xml -B test

  api-docker-build:
    runs-on: ubuntu-latest
    needs: [ api-tests ]
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Docker build (API)
        run: docker build -t mycellius-api:${{ github.sha }} ./api

  deploy-dev:
    runs-on: [ self-hosted, dev-runner ]
    needs: [ api-tests, api-docker-build ]
    if: github.event_name == 'push' && github.ref == 'refs/heads/dev'
    steps:
      - name: Ensure deploy script is executable
```

```
run: |
  cd /var/www/html/mycellius
  chmod +x infra/deploy/deploy.sh
- name: Deploy DEV
run: |
  cd /var/www/html/mycellius
  ./infra/deploy/deploy.sh dev DEV
```

deploy-test:

```
runs-on: [ self-hosted, test-runner ]
needs: [ api-tests, api-docker-build ]
if: github.event_name == 'push' && github.ref == 'refs/heads/test'
steps:
- name: Ensure deploy script is executable
  run: |
    cd /var/www/html/mycellius
    chmod +x infra/deploy/deploy.sh
- name: Deploy TEST
  run: |
    cd /var/www/html/mycellius
    ./infra/deploy/deploy.sh test TEST
```

Après avoir compléter ce fichier ci.yml, on va créer un runner Github Actions.

Un runner GitHub Actions, c'est la machine qui exécute concrètement tes jobs CI/CD.

Par défaut, GitHub fournit des runners "cloud" (ubuntu-latest), mais ils ne peuvent pas accéder à ton réseau Proxmox privé (192.168.x.x), donc impossible d'y faire des déploiements SSH.

Un runner "self-hosted" installé sur tes VMs exécute les jobs depuis TON réseau : il peut faire git pull, lancer docker compose, et tu obtiens un déploiement automatique DEV→TEST→PROD, reproductible et traçable (logs Actions = preuve).

Sans runners self-hosted, tu restes sur des déploiements manuels ou tu dois exposer tes VMs sur Internet (mauvaise idée en TP).

Ce qu'on va mettre en place

- 1 runner sur VM DEV (label dev-runner) pour exécuter deploy-dev
- 1 runner sur VM TEST (label test-runner) pour exécuter deploy-test
- 1 runner sur VM PROD (label prod-runner) pour exécuter deploy-prod (plus tard, quand tu valides la prod)

Étapes côté VM (DEV / TEST / PROD) – mêmes actions, seul le label change

Prérequis sur la VM :

- Git installé
- Docker installé (tu l'as déjà)
- Le repo est cloné (tu l'as déjà : /var/www/html/mycellius)

A) Créer le dossier runner (idéalement hors du repo, mais si tu restes dans le repo, ignore-le dans .gitignore)

Perso, je l'ai fait à la racine de projet mycellius mais pour une histoire de bonnes pratiques, il faudrait le faire à la racine du répertoire courant, mais on va le faire dans :

/var/www/html/mycellius/

et ajoute dans .gitignore à la racine du projet, dans la première ligne :

actions-runner/

Sinon ton deploy.sh avec git clean -fd peut le supprimer et je ne te raconte pas le bazar

B) Télécharger et extraire le runner sur chaque VM (commandes fournies par GitHub)

Étapes côté GitHub (interface)

- > Repository → Settings → Actions → Runners
- > "New self-hosted runner"
- > Choisir "Linux" + architecture (x64)
- > GitHub te donne une série de commandes (download + config). Tu ne les exécutes PAS sur ton PC, mais sur la VM cible (**connexion SSH en Putty ou powershell**). (perso j'avais 6 terminaux d'ouvert max. / 2 par VM)

C) Configurer le runner (commande fournie par GitHub)

Toujours sur la VM, dans le dossier du runner :

tu exécutes ./config.sh ... --token ...

quand il te demande le label, tu mets :

- DEV : dev-runner
- TEST : test-runner
- PROD : prod-runner (pas encore à faire)

D) Installer en service systemd (pour qu'il survive aux reboot)

```
sudo ./svc.sh install
sudo ./svc.sh start
sudo ./svc.sh status
```

=> Attendu : "active (running)".

E) Vérifier dans GitHub

Settings → Actions → Runners :

DEV : runner "Online / Idle" avec label dev-runner
TEST : runner "Online / Idle" avec label test-runner
PROD : runner "Online / Idle" avec label prod-runner

Étapes côté workflow (ci.yml) – lier chaque job au bon runner (votre fichier ci.yml est déjà OK)

But : éviter que deploy-dev parte sur TEST ou PROD.

Exemples :

Déploiement DEV (on fera pareil pour TEST et PROD (qu'on complètera plus tard)):

Ce qui est en caractère gras sera remplacé par test et prod...

```
deploy-dev:
  runs-on: [ self-hosted, dev-runner ]
  needs: [ api-tests, api-docker-build ]
  if: github.event_name == 'push' && github.ref == 'refs/heads/dev'
  steps:
    - name: Ensure deploy script is executable
      run: |
        cd /var/www/html/mycellius
        chmod +x infra/deploy/deploy.sh
    - name: Deploy DEV
      run: |
        cd /var/www/html/mycellius
        ./infra/deploy/deploy.sh dev DEV
```

Ce que tu fais VM par VM (résumé opérationnel)

- VM DEV : installer runner + label dev-runner + service start + vérifier "Idle"
- VM TEST : installer runner + label test-runner + service start + vérifier "Idle"
- VM PROD : idem label prod-runner (mais tu peux le faire plus tard, quand la chaîne dev→test est stable)

Ensuite, usage normal (preuve DEV→TEST→PROD)

- Push sur dev ⇒ deploy-dev se lance automatiquement sur la VM DEV.
- PR dev→test + merge ⇒ push sur test ⇒ deploy-test se lance automatiquement sur VM TEST.
- PR test→prod + merge ⇒ push sur prod ⇒ deploy-prod sur VM PROD.

Tester sur VM DEV et TEST

```
curl http://localhost:8080/api/health  
curl http://localhost:8080/api/version
```

Si jamais tu as ses résultats, suis le debug sinon passe à la suite directement :

```
clarisse@mycellius-dev:/var/www/html/mycellius$ curl http://localhost:8080/api/health
```

```
curl: (56) Recv failure: Connection reset by peer
```

```
clarisse@mycellius-dev:/var/www/html/mycellius$ curl http://localhost:8080/api/version
```

```
curl: (56) Recv failure: Connection reset by peer
```

```
clarisse@mycellius-dev:/var/www/html/mycellius$ docker logs --tail 120 mycellius-mysql
```

```
Fatal glibc error: CPU does not support x86-64-v2
```

La cause : l'image mysql:8.0 que tu utilises embarque une glibc compilée pour x86-64-v2, et ton CPU Proxmox ne supporte pas ce niveau d'instructions ⇒ MySQL ne peut pas démarrer, donc l'API crash (Flyway) et tes curl reset.

Tu n'as pas 50 solutions logicielles : il faut changer l'image DB (ou changer de CPU). Pour avancer vite en TP, la solution la plus simple est MariaDB (compatible MySQL pour ton usage).

Étape 1 — Modifier infra/stack/docker-compose.yml (dans le repo)

Remplace seulement l'image et les variables du service mysql (garde le nom de service mysql, sinon l'URL JDBC change).

remplace :

```
mysql:
  image: mysql:8.0
  container_name: mycellius-mysql
  restart: unless-stopped
  environment:
    MYSQL_ROOT_PASSWORD: rootpassword
    MYSQL_DATABASE: mycellius
    MYSQL_USER: mycellius
    MYSQL_PASSWORD: mycellius
  volumes:
    - ./data:/var/lib/mysql
```

par :

```
mysql:
  image: mariadb:10.11
  container_name: mycellius-mysql
  restart: unless-stopped
  environment:
    MARIADB_ROOT_PASSWORD: rootpassword
    MARIADB_DATABASE: mycellius
    MARIADB_USER: mycellius
    MARIADB_PASSWORD: mycellius
  volumes:
    - ./data:/var/lib/mysql
```

Tu ne touches pas au service api : jdbc:mysql://mysql:3306/... restera OK.

Étape 2 — Sur chaque VM (DEV puis TEST) : repartir proprement

```
cd /var/www/html/mycellius
docker compose -f infra/stack/docker-compose.yml down
sudo rm -rf infra/stack/data
sudo mkdir -p infra/stack/data

docker compose -f infra/stack/docker-compose.yml up -d mysql
docker logs --tail 50 mycellius-mysql
docker compose -f infra/stack/docker-compose.yml up -d api
docker logs --tail 50 mycellius-api

curl -i http://localhost:8080/api/health
```

Etape 3 — Commit + push sur DEV

Donc : fais d'abord le push du compose corrigé sur dev, puis on vérifie sur la VM DEV que MySQL (MariaDB) n'est plus en Restarting.

```
git checkout dev
git add infra/stack/docker-compose.yml
git commit -m "Use MariaDB to support older CPU (glibc x86-64-v2 issue)"
git push origin dev
```

Etape 4 — VM DEV : laisser le runner/deploy faire le pull + relance (ou le faire à la main si tu veux vérifier tout de suite)

Manuel (pour valider) :

```
cd /var/www/html/mycellius
./infra/deploy/deploy.sh dev DEV
docker ps --format "table {{.Names}}\t{{.Status}}"
docker logs --tail 30 mycellius-mysql
```

ou tu attends que le run fasse le pull DEV :

```
curl -i http://localhost:8080/api/health
```

Attendu :

```
clarisse@mycellius-dev:/var/www/html/mycellius$ curl -i http://localhost:8080/api/health
HTTP/1.1 200
Vary: Origin
Vary: Access-Control-Request-Method
Vary: Access-Control-Request-Headers
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: DENY
Referrer-Policy: no-referrer
Content-Type: application/json
Transfer-Encoding: chunked
Date: Tue, 24 Feb 2026 22:35:57 GMT
```

puis passe ce test :

```
curl -i http://localhost:8080/api/health
```

Résultats possibles :

> 200 avec JSON { "env": "...", "version": "..." } → parfait, preuve OK.

> 401 ou aucune réponse → normal si /api/version est protégé.

Dans ce cas, on le rend public (1 ligne dans SecurityConfig après :

```
.requestMatchers("/api/health", "/actuator/health").permitAll() :
```

mets :

```
.requestMatchers("/api/version").permitAll()
```

Après avoir ajouté `.requestMatchers("/api/version").permitAll()`, il faut redémarrer/redéployer l'API sur la VM pour que la config soit prise en compte.

```
git add api/src/main/java/fr/mycellius/security/SecurityConfig.java
git commit -m "Expose /api/version for deployment proof"
git push origin dev
```

Ensuite `curl -i http://localhost:8080/api/version` doit renvoyer 200 + JSON.

Attendu :

```
clarisse@mycellius-dev:/var/www/html/mycellius$ curl -i http://localhost:8080/api/version
HTTP/1.1 200
Vary: Origin
Vary: Access-Control-Request-Method
Vary: Access-Control-Request-Headers
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: DENY
Referrer-Policy: no-referrer
Content-Type: application/json
Transfer-Encoding: chunked
Date: Tue, 24 Feb 2026 23:18:45 GMT
```

BON POUR DEV, C'EST BOOOOOON !!!!!

Preuve attendue (objectif séance 11)

- Push sur dev : Actions vert + VM DEV à jour.
- PR dev → test puis merge la Pull Request pour que la VM test soit à jour :
 - + Actions vert
 - + VM TEST à jour
 - + GET /api/version sur TEST affiche la nouvelle version.

 Open [clakg](#) wants to merge 6 commits into [test](#) from [dev](#) 

 Conversation 0  Commits 6  Checks 4  Files changed 4



clakg commented 7 minutes ago

Owner ...

No description provided.



 **clakg** added 6 commits 10 hours ago

-   [Ignore actions-runner directory on VMs](#) ✗ 46fda69
-   [correction ci.yml + add chmod](#) ✓ dca7d24
-   [correction ci.yml + correction dev-runner to test-runner](#) ✓ 822b1a4
-   [Trigger deploy-dev](#) ✓ f16297e
-   [Use MariaDB to support older CPU \(glibc x86-64-v2 issue\)](#) ✓ 4cc6d97
-   [Expose /api/version for deployment proof](#) ✓ e66022f



All checks have passed

3 skipped, 5 successful checks

3 skipped checks ▾

-   CI / deploy-dev (pull_request) Skipped 6 minutes ago ...
-   CI / deploy-test (pull_request) Skipped 6 minutes ago ...
-   CI / deploy-test (push) Skipped 20 minutes ago ...

5 successful checks ▾

-   CI / api-docker-build (pull_request) Successful in 34s ...
-   CI / api-docker-build (push) Successful in 35s ...
-   CI / api-tests (pull_request) Successful in 19s ...



No conflicts with base branch

Merging can be performed automatically.

Merge pull request ▾

You can also merge this with the command line. [View command line instructions.](#)

Still in progress? [Convert to draft](#)

D) Release PROD + backup/restore validé + doc + démo finale

D1) Release prod (merge test→prod + tag)

Process minimal :

- PR test → prod (comme vos règles séance 1).
- Merge.
- Tag (ex v1.0.0) sur la branche prod (preuve “release”).

Option :

- déclencher deploy-prod sur push prod (même mécanique que deploy-test).

Règle simple :

- base = la branche qui reçoit (la cible)
- compare = la branche qui apporte les changements (la source)

Et pour rappel, la PR précédente est dev→test : base test, compare dev.

Dans ton process D1 (release PROD) :

PR test → prod

- > base = prod
- > compare = test

Ensuite, pour que GitHub Actions déploie PROD automatiquement :

ton workflow doit déclencher un job deploy-prod sur push de la branche prod (le merge crée un push sur prod).

Rajoute deploy-prod + la meme chose que pour dev et test + label prod-runner:

```
deploy-prod:
  runs-on: [ self-hosted, prod-runner ]
  needs: [ api-tests, api-docker-build ]
  if: github.event_name == 'push' && github.ref == 'refs/heads/prod'
  steps:
    - name: Ensure deploy script is executable
      run: |
        cd /var/www/html/mycellius
        chmod +x infra/deploy/deploy.sh

    - name: Deploy PROD
      run: |
        cd /var/www/html/mycellius
        ./infra/deploy/deploy.sh prod PROD
```

Je vous laisse faire un runner self-hosted sur la VM PROD avec le label prod-runner.
La VM PROD suit bien la branche prod dans /var/www/html/mycellius (comme DEV/TEST)

Côté runner PROD (étapes complètes)

Objectif : avoir un runner “self-hosted” sur la VM PROD avec le label prod-runner, pour que GitHub puisse exécuter deploy-prod dans le réseau Proxmox.

A) Sur GitHub

- Repo → Settings → Actions → Runners
- “New self-hosted runner”
- Choisir “Linux x64”
- GitHub te donne 3 blocs de commandes : download, extract, config (avec un token). Tu vas les exécuter sur la VM PROD.

B) Sur la VM PROD (SSH)

- Créer un dossier runner (recommandé hors du repo)

```
cd /var/www/html/mycellius/  
mkdir -p actions-runner  
cd actions-runner
```

- Exécuter les commandes “download + extract” données par GitHub (copier/coller).
- Tu verras apparaître run.sh, config.sh, svc.sh, runsvc.sh.
- Configurer le runner (commande GitHub ./config.sh ... --token ...)
- Pendant la config :
 - Runner name : ce que tu veux (ex mycellius-prod)
 - Labels : ajoute prod-runner (très important)
- Installer en service

```
sudo ./svc.sh install  
sudo ./svc.sh start  
sudo ./svc.sh status
```

C) Vérifier

- GitHub → Runners : le runner PROD doit être “Online / Idle” et afficher le label prod-runner.
- Ensuite (activation du déploiement)
- Commit + push sur dev de ton ci.yml modifié,
- puis PR dev→test → merge, (base: test ; compare: dev)
- puis PR test→prod → merge. (base: prod ; compare: test)
- Au merge test→prod, GitHub fait un push sur prod → deploy-prod se lance automatiquement.



All checks have passed

5 skipped, 5 successful checks

5 skipped checks

CI / deploy-dev (pull_request)

Skipped now

...

CI / deploy-dev (push)

Skipped 1 minute ago

...

CI / deploy-prod (pull_request)

Skipped now

...

CI / deploy-prod (push)

Skipped 1 minute ago

...

CI / deploy-test (pull_request)

Skipped now

...

5 successful checks

CI / api-docker-build (pull_request)

Successful in 32s

...

No conflicts with base branch

Merging can be performed automatically.

Merge pull request

You can also merge this with the command line. [View command line instructions.](#)

Still in progress? [Convert to draft](#)

← CI

✓ Merge pull request #25 from clakg/test #95

Re-run all jobs

Summary

All jobs

✓ api-tests

✓ api-docker-build

⌚ deploy-dev

⌚ deploy-test

✓ deploy-prod

Run details

Usage

Workflow file

Triggered via push 4 minutes ago

Status

Total duration

Artifacts

clakg pushed

f965fe0

prod

Success

3m 11s

—

ci.yml

on: push

✓ api-tests

14s

✓ api-docker-build

30s

⌚ deploy-dev

0s

✓ deploy-prod

2m 17s

⌚ deploy-test

0s

Note importante (pour éviter les ennuis)

Ne mets pas le runner dans /var/www/html/mycellius/actions-runner (dans le repo), sinon ton git clean -fd peut le supprimer. Si tu l'as déjà mis dans le repo, ajoute actions-runner/ dans .gitignore + commit/push.

46/59

Pour prouver qu'un déploiement PROD a bien fonctionné, tu fais 2 vérifications "preuves" (technique) + 1 vérification "visuelle" (web).

Preuve technique côté PROD (la plus fiable)

Sur la VM PROD :

```
curl -i http://localhost:8080/api/health
```

=> Attendu : HTTP 200.

Puis :

```
curl -i http://localhost:8080/api/version
```

=> Attendu : HTTP 200 + JSON avec env:"PROD" et version:"<hash>".

Et tu compares avec le commit déployé :

```
cd /var/www/html/mycellius  
git rev-parse --short HEAD
```

=> Le hash doit correspondre à version.

Preuve technique côté GitHub Actions

Sur GitHub → Actions → dernier run sur la branche prod :

> job deploy-prod en vert 

> dans les logs, tu vois "Deployed PROD / <hash>".

Vérification visuelle sur l'application web Mycellius (oui, possible)

Oui, mais seulement si ton front affiche une info d'environnement/version, ou si tu peux l'inférer.

Options "visuelles" :

A) Si tu as une page "À propos / Version" (ou un bandeau ENV) → tu vérifies qu'elle affiche PROD + la version.

B) Sinon, tu peux faire une vérif fonctionnelle :

- tu te connectes sur l'URL PROD du web
- tu ouvres une page wiki existante
- tu fais une action admin/dev attendue en prod (ex : consulter une page, créer/modifier si rôle autorisé)
- Ça prouve que l'ensemble (web → API → DB) fonctionne, mais ça ne prouve pas la version exacte.

Recommandation (rapide et propre) pour une preuve visuelle :

- Ajoute dans le front web, en bas de page (footer), l'affichage :

ENV et VERSION récupérés via GET /api/version

Comme ça, tu as une preuve visuelle immédiate en prod (et en test/dev).

D2) Backup/restore VALIDÉ

Votre procédure “de base” est déjà en séance 5.

En séance 12, la preuve attendue c’est : **“je sais restaurer” (CIA, disponibilité/intégrité)**.

Procédure robuste (sur VM DEV ou TEST, pas PROD)

1. Vérifier le nom du conteneur :

```
docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Status}}"
```

→ vous devez voir mycellius-mysql.

2. Faire le dump (dans un dossier backups)

```
mkdir -p /srv/mycellius/infra/db/backups
```

```
docker exec -i mycellius-mysql mysqldump \  
-u root -prootpassword \  
--single-transaction --routines --triggers \  
mycellius > /srv/mycellius/infra/db/backups/mycellius_backup.sql
```

3. Test de restauration (le “piège” classique : restaurer pendant que l’app écrit)

– Stoppez l’API (recommandé) :

```
docker stop mycellius-api
```

– Drop/Create DB :

```
docker exec -i mycellius-mysql mysql -u root -prootpassword -e "DROP DATABASE  
mycellius; CREATE DATABASE mycellius;"
```

– Restore :

```
docker exec -i mycellius-mysql mysql -u root -prootpassword mycellius < /srv/mycellius/  
infra/db/backups/mycellius_backup.sql
```

– Redémarrer l’API :

```
docker start mycellius-api
```

4. Preuve : GET d’une page existante (ou liste) OK + migrations Flyway cohérentes.

Les 4 causes les plus fréquentes de “restore KO”:

- Vous n’êtes pas dans le bon conteneur (nom différent de mycellius-mysql).
- Vous restaurez dans la mauvaise DB (ex : mysql au lieu de mycellius).
- L’API tourne et rejoue Flyway / écrit pendant le restore (d’où le stop API).
- Vous avez dumpé vide (fichier 0 octet) parce que la redirection > a été faite dans un mauvais shell/dossier.

D3) Docs + démo scénarisée (livrables “E5”)

1. Docs (2 fichiers Markdown suffisent)

– docs/10_user_guide.md :

- > login,
- > consulter,
- > rechercher,
- > créer/éditer (selon rôle),
- > voir révisions.

– docs/11_admin_guide.md :

- > déploiement (branches),
- > pipeline,
- > secrets,
- > audit,
- > backup/restore,
- > procédures incident.

2. Démo finale (3 rôles, 5 minutes chrono)

Scénario demandé : “**stagiaire consulte, dev publie, admin gère**”.

- **Stagiaire** : login → liste → ouvre une page → recherche.
- **Dev** : modifie une page → prouve qu’une révision est créée → audit log montre l’action.
- **Admin** : consulte audit + lance (ou montre) la procédure backup/restore validée.

Ca c'est du plus mais ça me paraît important de savoir livrer une application web en production, ce n'est pas une chose qui est pratiquée mais j'estime que c'est une belle finalité pour votre travail et investissement dans ce projet pour l'intégrer dans vos CVs.

Le but maintenant : donner aussi une IP Tailscale à la VM PROD, puis accéder au site web via cette IP 100.x (au lieu de 192.168.1.53).

Accéder au WEB React servi par nginx sur PROD)

A) Pré-requis : sur l'interface tailscale :

1. créer un Host alias "mycellius-prod"

- Dans Tailscale Admin → Access controls → onglet Hosts
- Ajoute une entrée :
 - > Host : mycellius-prod
 - > IP : 100.66.190.108
- Sauvegarde.
- Retourne sur General access rules → Add rule
 - > Source : autogroup:members (ou ton user)
 - > Destination : mycellius-prod
 - > Port/protocol : tcp:80, tcp:8080
- Save

2. En JSON (si tu préfères), ça donne :

```
{
  "src": ["autogroup:member"],
  "dst": ["host:mycellius-prod"],
  "ip": ["tcp:80", "tcp:8080", "tcp:443"],
}
```

B) Sur la VM PROD : installer Tailscale

1. SSH sur la VM PROD, puis :

```
curl -fsSL https://tailscale.com/install.sh | sh
```

2. Démarrer et associer la VM à ton compte :

```
sudo tailscale up
```

- Cette commande affiche une URL "login".

- Copie-colle l'URL dans ton navigateur (sur ton PC), connecte-toi à Tailscale, et valide l'ajout de la machine.

<https://login.tailscale.com/a/175dccad0105ed>

3. Vérifier la connexion + récupérer l'IP Tailscale (100.x) de la VM PROD :

```
clarisse@mycellius-prod:/var/www/html/mycellius$ tailscale status
```

```
100.66.190.108 mycellius-prod clakg35@ linux offline
100.69.198.124 ckg-formation clakg35@ windows -
100.99.254.80 iphone-12 clakg35@ iOS offline, last seen 9d ago
100.74.82.56 iphone-13 clakg35@ iOS offline, last seen 3d ago
100.114.143.17 px1 tagged-devices linux -
100.117.246.76 redmi-pad-pro clakg35@ android offline, last seen 5d ago
```

```
# Health check:
```

```
# - Some peers are advertising routes but --accept-routes is false
```

```
clarisse@mycellius-prod:/var/www/html/mycellius$ tailscale ip -4
```

```
100.66.190.108
```

4. Vérifier que le service Tailscale tourne : sudo systemctl status tailscaled --no-pager

```
clarisse@mycellius-prod:/var/www/html/mycellius$ sudo systemctl status tailscaled --no-pager
```

```
● tailscaled.service - Tailscale node agent
```

```
Loaded: loaded (/usr/lib/systemd/system/tailscaled.service; enabled; preset: enabled)
```

```
Active: active (running) since Wed 2026-02-25 20:54:17 UTC; 11min ago
```

```
Docs: https://tailscale.com/kb/
```

```
Main PID: 20721 (tailscaled)
```

```
Status: "Connected; clakg35@gmail.com; 100.66.190.108 fd7a:115c:a1e0::9a39:be6c"
```

```
Tasks: 11 (limit: 4602)
```

```
Memory: 15.7M (peak: 16.4M)
```

```
CPU: 2.024s
```

```
CGroup: /system.slice/tailscaled.service
```

```
└─20721 /usr/sbin/tailscaled --state=/var/lib/tailscale/tailscaled.state --...
```

```
févr. 25 20:57:38 mycellius-prod tailscaled[20721]: wgengine: Reconfig: user dialer
```

```
févr. 25 20:57:38 mycellius-prod tailscaled[20721]: tsdial: bart table size: 8
```

```
févr. 25 20:57:38 mycellius-prod tailscaled[20721]: wgengine: Reconfig: configuring DNS
```

```
févr. 25 20:57:38 mycellius-prod tailscaled[20721]: dns: Set: {DefaultResolvers:[] R...:6}
```

```
févr. 25 20:57:38 mycellius-prod tailscaled[20721]: dns: Resolvercfg: {Routes:{ts.ne...pa}
```

```
févr. 25 20:57:38 mycellius-prod tailscaled[20721]: dns: OScfg: {Nameservers:[100.10...pa}
```

févr. 25 20:57:38 mycellius-prod tailscaled[20721]: Switching ipn state Starting -> ...ue)
févr. 25 20:59:26 mycellius-prod tailscaled[20721]: Received error: PollNetMap: unex...EOF
févr. 25 20:59:39 mycellius-prod tailscaled[20721]: control: netmap: got new dial pl...rol
févr. 25 20:59:39 mycellius-prod tailscaled[20721]: health(warnable=not-in-map-poll): ok
Hint: Some lines were ellipsized, use -l to show in full.

5. Si ce n'est pas "active (running)", démarre-le et active-le :

```
sudo systemctl enable --now tailscaled  
sudo systemctl restart tailscaled  
sudo systemctl status tailscaled --no-pager
```

6. Reconnecter la VM à la tailnet (si nécessaire)

clarisse@mycellius-prod:/var/www/html/mycellius\$ sudo tailscale up
Some peers are advertising routes but --accept-routes is false

(Si une URL de login s'affiche, tu la valides comme tout à l'heure.)

7. Vérifier qu'elle est bien online + récupérer l'IP

clarisse@mycellius-prod:/var/www/html/mycellius\$ tailscale status
100.66.190.108 mycellius-prod clakg35@ linux -
100.69.198.124 ckg-formation clakg35@ windows -
100.99.254.80 iphone-12 clakg35@ iOS offline, last seen 9d ago
100.74.82.56 iphone-13 clakg35@ iOS offline, last seen 3d ago
100.114.143.17 px1 tagged-devices linux -
100.117.246.76 redmi-pad-pro clakg35@ android offline, last seen 5d ago

Health check:
- Some peers are advertising routes but --accept-routes is false

clarisse@mycellius-prod:/var/www/html/mycellius\$ tailscale ip -4
100.66.190.108

8. Ouvrir le firewall pour l'interface Tailscale (UFW)

```
sudo ufw allow in on tailscale0 to any port 80 proto tcp  
sudo ufw allow in on tailscale0 to any port 8080 proto tcp  
sudo ufw reload
```

9. Une fois fini, teste depuis ton PC :

<http://100.66.190.108/>
<http://100.66.190.108:8080/api/health>

C) Pour voir ton React en PROD, il faut :

- construire le front (web/dist)
- copier le build dans /var/www/html (ou configurer nginx pour pointer vers ce dossier)

1) Build du front React (sur la VM PROD)

Depuis /var/www/html/mycellius :

```
cd /var/www/html/mycellius/web  
npm ci  
npm run build
```

Attendu : un dossier dist/ est créé.

2. Déployer le build dans nginx

```
sudo rm -rf /var/www/html/*  
sudo cp -r dist/* /var/www/html/  
sudo chown -R www-data:www-data /var/www/html  
sudo systemctl reload nginx
```

3. Test local VM PROD

```
curl -I http://localhost/
```

Attendu : 200 et Last-Modified doit être “maintenant” (plus Dec 2025).

4. Test depuis ton PC via Tailscale

<http://100.66.190.108/>

Important (API URL)

Ton React en prod doit appeler l’API.

Si en dev tu appelais <http://localhost:8080>, en prod il faut appeler l’API de la VM :

soit <http://100.66.190.108:8080>
soit un reverse-proxy nginx /api -> 8080 (mieux)

Si probleme de connexion sur la page de login web :

1. Corriger le front : ne plus fallback sur localhost

Dans web/src/api/apiClient.js, remplace la ligne :

```
const API_URL = import.meta.env.VITE_API_URL ?? 'http://localhost:8080';
```

par :

```
const API_URL = import.meta.env.DEV
  ? (import.meta.env.VITE_API_URL ?? "http://localhost:8080")
  : "";
```

Ainsi, en prod, si tu ne mets pas VITE_API_URL, il appellera /api/v1/... sur le même host (port 80).

2. Copie de TOUT web/dist/ vers /var/www/html/

Le “niveau” logique :

Source (le build front déjà prêt) :

/home/clarisse/actions-runner-prod/_work/mycellius-ckg/mycellius-ckg/web/dist/

Destination (répertoire servi par Nginx) :

/var/www/html/

source = dist généré par le runner

SRC="/home/clarisse/actions-runner-prod/_work/mycellius-ckg/mycellius-ckg/web/dist/"

DST="/var/www/html/"

```
sudo rsync -a --exclude 'app/' "$SRC" "$DST"
```

vérifs

```
sudo ls -la /var/www/html/assets | head -n 20
```

```
sudo test -f /var/www/html/assets/index-OBqCUrN_.js && echo "OK: JS copié" || echo "KO: JS absent"
```

```
curl -I http://localhost/assets/index-OBqCUrN_.js
```

3. Correction des fichiers de déploiement en local pour éviter de reproduire ces erreurs :

a- docker-compose.yml

b- deploy.sh

c- ci.yml

a- docker-compose.yml

> Dans docker-compose.yml (infra/stack/docker-compose.yml):

Remplace :

```
volumes:  
  - ./data:/var/lib/mysql
```

par:

```
volumes:  
  - mycellius_db:/var/lib/mysql
```

> Supprime le "*environnement*" en double si tu l'as

> Rajoute au même niveau que "*version*" et "*services*":

```
volumes:  
  mycellius_db:
```

b- deploy.sh

remplace le contenu actuel par (ce qui en gras est ce qui est rajouté):

```
#!/usr/bin/env bash
set -euo pipefail

BRANCH="${1:-dev}"    # dev | test | prod
ENV_NAME="${2:-DEV}"  # DEV | TEST | PROD

# Dossier de déploiement (repo cloné "stable") :
# - tu peux le forcer via DEPLOY_DIR (recommandé dans CI)
# - sinon auto-détection (DEV/TEST vs PROD)
DEPLOY_DIR="${DEPLOY_DIR:-}"

if [[ -z "$DEPLOY_DIR" ]]; then
    if [[ -d "/var/www/html/mycellius/.git" ]]; then
        DEPLOY_DIR="/var/www/html/mycellius"
    elif [[ -d "/opt/mycellius/.git" ]]; then
        DEPLOY_DIR="/opt/mycellius"
    else
        echo "ERROR: Impossible de trouver le repo de déploiement."
        echo "Attendu: /var/www/html/mycellius/.git ou /opt/mycellius/.git"
        echo "Fix: exporte DEPLOY_DIR=/chemin/du/repo avant d'exécuter le script."
        exit 1
    fi
fi

echo "[deploy] DEPLOY_DIR=$DEPLOY_DIR BRANCH=$BRANCH ENV=$ENV_NAME"

cd "$DEPLOY_DIR"

# Sécurité : on doit être dans un repo Git
git rev-parse --is-inside-work-tree >/dev/null

# Sync propre sur la branche distante
git fetch origin
git checkout -B "$BRANCH" "origin/$BRANCH"
git reset --hard "origin/$BRANCH"
git clean -fd

export MYCELLIUS_ENV="$ENV_NAME"
export MYCELLIUS_VERSION="$(git rev-parse --short HEAD)"

COMPOSE_FILE="infra/stack/docker-compose.yml"
if [[ ! -f "$COMPOSE_FILE" ]]; then
    echo "ERROR: Compose introuvable: $DEPLOY_DIR/$COMPOSE_FILE"
    exit 1
fi

# Force un nom de projet stable (évite les surprises si le dossier change)
COMPOSE_PROJECT_NAME="${COMPOSE_PROJECT_NAME:-stack}"

docker compose -p "$COMPOSE_PROJECT_NAME" -f "$COMPOSE_FILE" up -d --build

echo "[deploy] Deployed $ENV_NAME / $MYCELLIUS_VERSION"
```

c. ci.yml

Remplace par ce contenu mis à jour (on voit ensemble ce qui a changé) :

```
name: CI

on:
  push:
    branches: [ "dev", "test", "prod" ]
  pull_request:
    branches: [ "test", "prod" ]

jobs:
  api-tests:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Java 21
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: "21"
          cache: maven

      - name: Build and run tests for API
        run: mvn -f api/pom.xml -B test

  api-docker-build:
    runs-on: ubuntu-latest
    needs: [ api-tests ]
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Docker build (API)
        run: docker build -t mycellius-api:${{ github.sha }} ./api

  deploy-dev:
    runs-on: [ self-hosted, dev-runner ]
    needs: [ api-tests, api-docker-build ]
    if: github.event_name == 'push' && github.ref == 'refs/heads/dev'
    steps:
      - uses: actions/checkout@v4

      - name: Ensure deploy script is executable
        run: chmod +x infra/deploy/deploy.sh

      - name: Deploy DEV
        env:
          DEPLOY_DIR: /var/www/html/mycellius
          COMPOSE_PROJECT_NAME: stack
        run: ./infra/deploy/deploy.sh dev DEV

  deploy-test:
    runs-on: [ self-hosted, test-runner ]
    needs: [ api-tests, api-docker-build ]
    if: github.event_name == 'push' && github.ref == 'refs/heads/test'
    steps:
      - uses: actions/checkout@v4

      - name: Ensure deploy script is executable
```

```

run: chmod +x infra/deploy/deploy.sh

- name: Deploy TEST
  env:
    DEPLOY_DIR: /var/www/html/mycellius
    COMPOSE_PROJECT_NAME: stack
  run: ./infra/deploy/deploy.sh test TEST

deploy-prod:
  runs-on: [ self-hosted, prod-runner ]
  needs: [ api-tests, api-docker-build ]
  if: github.event_name == 'push' && github.ref == 'refs/heads/prod'
  steps:
    - uses: actions/checkout@v4

    - name: Ensure deploy script is executable
      run: chmod +x infra/deploy/deploy.sh

    - name: Deploy PROD
      env:
        DEPLOY_DIR: /opt/mycellius
        COMPOSE_PROJECT_NAME: stack
      run: ./infra/deploy/deploy.sh prod PROD

deploy-web-prod:
  runs-on: [ self-hosted, prod-runner ]
  needs: [ api-tests, api-docker-build ]
  if: github.event_name == 'push' && github.ref == 'refs/heads/prod'
  steps:
    - uses: actions/checkout@v4

    - name: Who am I (debug)
      run: whoami

    - name: Build React (web)
      working-directory: web
      run: |
        npm ci
        npm run build

    - name: Stage build (owned by clarisse)
      run: |
        mkdir -p /var/www/deploy/mycellius-web
        rm -rf /var/www/deploy/mycellius-web/*
        cp -r web/dist/* /var/www/deploy/mycellius-web/

    - name: Publish to nginx (single sudo)
      run: |
        sudo rsync -a --delete /var/www/deploy/mycellius-web/ /var/www/html/
        sudo chown -R www-data:www-data /var/www/html
        sudo systemctl reload nginx

```

Voilà, normalement, ton application web est déployée sur la VM PROD et est opérationnelle !!!!